

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

Сыктывкарский лесной институт (филиал) федерального  
государственного бюджетного образовательного учреждения  
высшего образования «Санкт-Петербургский государственный  
лесотехнический университет имени С. М. Кирова»  
(СЛИ)

РУКОВОДСТВО  
АДМИНИСТРАТОРА

CMS для парикмахерских и барбершопов «Сайтама»

Исполнитель

В. А. Решетняк

Руководитель практики

Н. А. Оганезова

Сыктывкар 2026

## **АННОТАЦИЯ**

Настоящий документ – руководство администратора программного продукта «CMS для парикмахерских и барбершопов «Сайтама»» (далее – система, CMS «Сайтама») по развёртыванию и эксплуатации серверной части системы.

Документ адресован техническому специалисту, отвечающему за установку, настройку, обновление и сопровождение системы на сервере, – и не описывает работу с пользовательским интерфейсом (запись клиентов, управление услугами, отчёты и т. п.). Эти сведения приведены в отдельном руководстве пользователя (см. «Введение»).

Документ подготовлен в соответствии со структурой, предусмотренной ГОСТ 19.105-78, на основе фактического состава развёртывания системы на момент подготовки документа (2026 год).

## СОДЕРЖАНИЕ

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>АННОТАЦИЯ</b> . . . . .                                           | <b>2</b>  |
| <b>ВВЕДЕНИЕ</b> . . . . .                                            | <b>4</b>  |
| <b>1 Назначение программы</b> . . . . .                              | <b>5</b>  |
| <b>2 Условия выполнения программы</b> . . . . .                      | <b>6</b>  |
| Состав технических и программных средств . . . . .                   | 6         |
| Требования к квалификации пользователя . . . . .                     | 6         |
| Условия применения . . . . .                                         | 7         |
| <b>3 Подготовка к работе</b> . . . . .                               | <b>8</b>  |
| 3.1 Развёртывание через Docker Compose (рекомендуемый способ)        | 8         |
| 3.2 Подключение домена и HTTPS . . . . .                             | 9         |
| 3.3 Настройка почты (SMTP) и входа через VK ID . . . . .             | 9         |
| 3.4 Первичная настройка приложения . . . . .                         | 10        |
| 3.5 Проверка работоспособности . . . . .                             | 10        |
| <b>4 Описание операций</b> . . . . .                                 | <b>12</b> |
| 4.1 Обновление развёрнутой версии . . . . .                          | 12        |
| 4.2 Резервное копирование и восстановление базы данных . . . . .     | 12        |
| 4.3 Мониторинг состояния . . . . .                                   | 13        |
| 4.4 Управление переменными окружения . . . . .                       | 14        |
| 4.5 Безопасность инфраструктуры . . . . .                            | 14        |
| <b>5 Сообщения оператору</b> . . . . .                               | <b>16</b> |
| Сообщения при запуске и настройке . . . . .                          | 16        |
| Сообщения при развёртывании и работе контейнеров . . . . .           | 17        |
| Действия при сбоях и восстановлении . . . . .                        | 18        |
| <b>ПРИЛОЖЕНИЕ А. Переменные окружения</b> . . . . .                  | <b>20</b> |
| <b>ПРИЛОЖЕНИЕ Б. Состав контейнеров</b> . . . . .                    | <b>23</b> |
| <b>ПРИЛОЖЕНИЕ В. Глоссарий терминов</b> . . . . .                    | <b>24</b> |
| <b>ПРИЛОЖЕНИЕ Г. Пример развёртывания на новом сервере</b> . . . . . | <b>26</b> |

## ВВЕДЕНИЕ

**Область применения документа.** Документ описывает установку и эксплуатацию серверной части CMS «Сайтама»: состав компонентов развёртывания, требования к серверу, порядок первого запуска, настройку переменных окружения, обновление версии, резервное копирование и восстановление данных, мониторинг состояния и типовые нештатные ситуации. Документ не описывает бизнес-логику и пользовательский интерфейс системы.

**Перечень эксплуатационной документации.** Работа с пользовательским интерфейсом (личный кабинет клиента и мастера, панель администратора точки и владельца сети) описана в отдельном руководстве пользователя. Программный интерфейс (API) документирован автоматически средствами FastAPI в формате OpenAPI/Swagger и доступен по адресу `/api/docs` развёрнутого экземпляра системы. Настоящий документ самодостаточен для задач установки и сопровождения.

**Уровень подготовки пользователя.** Документ рассчитан на читателя с базовыми навыками работы в командной строке Linux, начальным опытом работы с Docker и Docker Compose, общим представлением о переменных окружения и настройке DNS-записей домена. Для операций резервного копирования/восстановления и для запуска системы без Docker дополнительно полезны базовые навыки работы с PostgreSQL (`psql`, `pg_dump`).

## 1 Назначение программы

CMS «Сайтама» – веб-приложение для автоматизации онлайн-записи клиентов и операционной деятельности сети парикмахерских/барбершопов; назначение и функции системы с точки зрения пользователей подробно описаны в руководстве пользователя (раздел 1 того документа). Настоящий раздел описывает систему со стороны того, что администратор фактически разворачивает и поддерживает на сервере.

Система разворачивается как набор взаимодействующих контейнеров Docker: веб-сервер с TLS-терминацией и security-заголовками (Caddy), контейнер фронтенда (отдаёт собранное веб-приложение и проксирует запросы к API), контейнер бэкенда (FastAPI/Python) и сервер базы данных (PostgreSQL), а также контейнер автоматического резервного копирования. Ниже приведена общая схема состава развёртывания.

- 1) Браузер клиента обращается к **Caddy** (TLS-терминация, порты 80/443, автоматический сертификат Let's Encrypt).
- 2) Caddy передаёт запрос контейнеру frontend (nginx: отдаёт собранное веб-приложение и проксирует запросы /api/\* дальше).
- 3) Контейнер frontend проксирует /api/\* на контейнер backend (FastAPI/uvicorn, порт 8000).
- 4) Контейнер backend обращается к контейнеру db (PostgreSQL 16, порт 5432).
- 5) Рядом с db работает контейнер backup-local (ежедневный pg\_dump в ./backups) и опциональный backup-s3.

Наружу из этой топологии открыты только порты 80 и 443 контейнера Caddy – порты базы данных, бэкенда и фронтенда в продакшен-конфигурации (docker-compose.yml) на хост не публикуются, обращение к ним возможно только внутри внутренней docker-сети.

## 2 Условия выполнения программы

### Состав технических и программных средств

- **Сервер (VPS) или иной хост с Linux**, на котором установлены Docker Engine и плагин Docker Compose (команда `docker compose`, без дефиса). Рекомендуемая конфигурация (техническое задание, раздел 5.4): от 2 CPU, 4 GB RAM, 40 GB SSD – оценочная величина, обоснованная нагрузочным тестом (см. ТЗ, раздел 7.1); этого достаточно для одновременной работы шести контейнеров (PostgreSQL, backend, frontend, Caddy, ежедневный бэкап и, опционально, S3-бэкап). Перед вводом в эксплуатацию всё равно рекомендуется повторить нагрузочное тестирование на целевом сервере и уточнить конфигурацию по факту.
- **Открытые наружу порты 80 и 443** (HTTP/HTTPS через Caddy). Остальные порты наружу открывать не нужно и небезопасно.
- **Домен с А-записью**, указывающей на IP сервера – обязателен для выпуска TLS-сертификата (Let's Encrypt не выдаёт сертификаты по IP-адресу). До появления домена система работает по обычному HTTP на голом IP.
- **Настоящий HTTPS-домен обязателен для входа через VK ID** – по IP или localhost в проде эта функция не работает.
- Для запуска без Docker (только для разработки/тестирования, не рекомендуется для прод-эксплуатации): Python 3.14, Node.js 18+, PostgreSQL 13+ с правами на CREATE EXTENSION (нужно расширение btree\_gist).
- Для просмотра/восстановления резервных копий – программы gunzip и psql (входят в стандартный образ PostgreSQL, используемый в docker compose exec).

### Требования к квалификации пользователя

Базовые навыки работы в командной строке Linux и с Docker/Docker Compose; понимание назначения переменных окружения; начальные знания настройки DNS-записей домена. Для восстановления БД из резервной копии и для запуска системы без Docker – базовые навыки работы с PostgreSQL.

## Условия применения

Система рассчитана на постоянное подключение сервера к Интернету и на работу **ровно одного** экземпляра контейнера backend – защита от перебора паролей, планировщик напоминаний о записи и ограничение частоты запросов реализованы внутри процесса бэкенда, без внешней очереди задач или общего хранилища состояния между экземплярами. Горизонтальное масштабирование бэкенда (несколько одновременно работающих реплик) в текущей версии системы не поддерживается.

Требования к надёжности (ТЗ, раздел 4.1.3): среднее время наработки на отказ – не менее 200 часов непрерывной работы в период опытной эксплуатации; время восстановления после сбоя – не более 30 минут (см. раздел «Действия при сбоях...»).

### 3 Подготовка к работе

#### 3.1 Развёртывание через Docker Compose (рекомендуемый способ)

1) Установите Docker, если он ещё не установлен:

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER && newgrp docker
```

2) Скопируйте код на сервер и перейдите в каталог проекта:

```
git clone https://github.com/Stonuv/fastapi_hair_salon
cd fastapi_hair_salon
cp .env.example .env
```

3) Заполните в файле .env обязательные значения SECRET\_KEY и SETUP\_TOKEN – без них контейнер backend откажется запускаться (см. 5). Команды генерации приведены прямо в комментариях файла:

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))" # SECRET_KEY
python3 -c "import secrets; print(secrets.token_urlsafe(24))" # SETUP_TOKEN
```

Остальные переменные (SITE\_ADDRESS, COOKIE\_SECURE, SMTP\_\*, VK\_\*) можно оставить закомментированными – система заработает по IP на голом HTTP. Полный перечень – приложение А.

4) Откройте необходимые порты в брандмауэре:

```
sudo ufw allow OpenSSH
sudo ufw allow 80,443/tcp
sudo ufw enable
```

5) Запустите стек:

```
docker compose up --build -d
```

Миграции базы данных накатываются автоматически при старте контейнера backend, отдельного шага не требуется.

6) Откройте <http://<IP сервера>/setup> и введите значение SETUP\_TOKEN – это создаёт первого владельца сети и первую точку

сети (подробный пошаговый разбор мастера настройки – в руководстве пользователя, раздел 3.1). После этого /setup самоблокируется – повторно им воспользоваться нельзя.

### 3.2 Подключение домена и HTTPS

Когда домен готов (А-запись указывает на IP сервера):

1) Впишите в .env:

```
SITE_ADDRESS=your-domain.ru  
COOKIE_SECURE=true  
FRONTEND_BASE_URL=https://your-domain.ru
```

2) Перезапустите стек: `docker compose up -d` – Caddy сам получит сертификат Let's Encrypt и включит редирект `http -> https`, без ручной правки Caddyfile.

После этого можно донстроить вход через VK ID (redirect URI приложения и `VK_REDIRECT_URI` – на `https://your-domain.ru/api/auth/vk/callback`) и подключить боевой SMTP-провайдер (см. 3.3).

### 3.3 Настройка почты (SMTP) и входа через VK ID

**Почта.** Без заполненных `SMTP_*` переменных попытки отправки писем (подтверждение почты, восстановление пароля, напоминания, перенос записи) молча падают в лог контейнера backend – само приложение при этом не ломается. Для боевого провайдера заполните `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_USE_TLS`, `SMTP_FROM_EMAIL`, `SMTP_FROM_NAME` в .env (пример для Яндекс.Почты – `smtp.yandex.ru:465`, `SMTP_USE_TLS=true`, пароль – «пароль приложения», `id.yandex.ru/security`, а не обычный пароль от почты).

**Локальная проверка почты (только для разработки).** Поднимите Mailpit – фейковый SMTP-сервер, перехватывающий письма вместо реальной отправки:

```
docker compose -f docker-compose.dev.yml up -d
```

Дефолты в backend/.env.example уже нацелены на него (SMTP\_HOST=localhost, SMTP\_PORT=1025); письма смотрите на http://localhost:8025.

**VK ID.** Требуется зарегистрированное приложение на id.vk.com/business: создайте приложение, укажите redirect URI https://<домен>/api/auth/vk/callback (в проде – только настоящий HTTPS-домен, не IP и не localhost) и впишите client\_id/client\_secret в VK\_CLIENT\_ID/ VK\_CLIENT\_SECRET. Без VK\_CLIENT\_ID кнопка «Войти через VK» на сайте автоматически скрывается – это ожидаемое поведение, а не ошибка.

### 3.4 Первичная настройка приложения

Создание первого владельца сети и первой точки сети выполняется через мастер /setup сразу после запуска стека (см. шаг 6 в 3.1). Полный пошаговый разбор этого мастера (все четыре шага, поля формы, подсказки) приведён в руководстве пользователя, раздел 3.1 – с точки зрения администратора здесь важно только то, что до этого момента сайт недоступен для обычных посетителей, а после – мастер настройки самоблокируется навсегда.

### 3.5 Проверка работоспособности

1) Убедитесь, что все контейнеры запущены и здоровы:

```
docker compose ps
```

У каждого сервиса со своим healthcheck (db, backend, frontend, caddy) в колонке состояния должно быть healthy. Запуск идёт по цепочке зависимостей: backend стартует только после того, как db стал healthy; frontend – после backend; caddy – после frontend.

2) Проверьте состояние приложения напрямую:

```
curl http://<IP или домен>/api/./health # либо откройте в браузере
```

Ожидаемый ответ – {"status": "ok"}. Ответ 503 со сообщением "database unavailable" означает, что бэкенд поднялся, а база данных – нет (см. 5).

3) Откройте сайт в браузере и убедитесь, что главная страница

загружается, а `/api/docs` показывает документацию API.

## 4 Описание операций

### 4.1 Обновление развёрнутой версии

Автоматического развёртывания по push/merge в проекте нет – обновление выполняется вручную на сервере одной командой:

```
./scripts/deploy.sh
```

- 1) `git pull --ff-only`.
- 2) `docker compose up --build -d --wait --wait-timeout 120`.
- 3) Ожидание healthcheck всех сервисов.
- 4) Успех – либо ненулевой код завершения при неудаче (сломанный деплой не остаётся молча висеть).

Скрипт выполняет `git pull --ff-only`, пересобирает и перезапускает стек и дожидается healthcheck-статуса backend/frontend/caddy, прежде чем считать обновление успешным; если после пересборки какой-то из сервисов не стал healthy – скрипт завершается с ошибкой. Миграции базы данных накатываются автоматически при старте контейнера backend, отдельного шага не требуется. Отката (rollback) на предыдущую версию скрипт не выполняет – это осознанное ограничение (см. также раздел «Известные ограничения» README.md проекта).

### 4.2 Резервное копирование и восстановление базы данных

**Автоматическое локальное копирование.** Контейнер backup-local поднимается сразу вместе с остальным стеком, без дополнительной настройки: ежедневный сжатый pg\_dump в каталог ./backups рядом с docker-compose.yml, с ротацией – 7 ежедневных, 4 еженедельных, 6 ежемесячных копий (значения BACKUP\_KEEP\_DAYS/BACKUP\_KEEP\_WEEKS/BACKUP\_KEEP\_MONTHS в docker-compose.yml). Каталог ./backups смонтирован с диска сервера (не именованный том Docker), поэтому копии видны и доступны для передачи на другую машину (rsync/scp) – сам по себе локальный бэкап не защищает от потери всего сервера или его диска целиком.

**Дополнительное копирование в S3 (опционально).** Второй, независимый бэкап во внешнее S3-совместимое хранилище (Yandex Object Storage, Selectel, Backblaze B2, AWS S3 и т.п.) переживает потерю самого сервера:

1) заполните S3\_BACKUP\_\* переменные в .env (endpoint, регион, бакет, ключи доступа);

2) запустите: `docker compose --profile s3-backup up -d`.

Без заполненных переменных контейнер backup-s3 не запускается вовсе (профиль s3-backup по умолчанию выключен).

### **Восстановление из резервной копии.**

1) Найдите самый свежий дамп:

```
ls -t backups/barbershop/daily/*.sql.gz | head -1
```

2) Остановите бэкенд (чтобы он не писал в базу во время восстановления):

```
docker compose stop backend
```

3) Восстановите базу из дампа:

```
gunzip -c backups/barbershop/daily/barbershop-<дата>.sql.gz | \
docker compose exec -T db psql -U barbershop -d barbershop
```

4) Запустите бэкенд обратно:

```
docker compose start backend
```

## **4.3 Мониторинг состояния**

– `docker compose ps` – быстрый обзор состояния всех контейнеров (running/healthy или нет).

– `docker compose logs <сервис>` (например, `docker compose logs backend`) – журнал конкретного контейнера; добавьте `-f` для просмотра в реальном времени.

– `GET /health` – проверка самого приложения и его связи с базой данных (используется healthcheck'ом контейнера backend и подходит для

внешних систем мониторинга): 200 {"status": "ok"} в норме, 503 {"detail": "database unavailable"}, если приложение не может выполнить простой запрос к базе.

– **Мониторинг ошибок (Sentry) подготовлен, но не подключён.**

Код инициализации есть и на бэкенде, и на фронтенде, но без указанного DSN `sentry_sdk.init(...)` не вызывается – мониторинг выключен целиком. Когда появится доступ к `sentry.io` (или self-hosted/GlitchTip-инстансу с DSN того же формата):

- 1) создайте проект, скопируйте DSN;
- 2) впишите в `.env`: `SENTRY_DSN=...` (бэкенд) и `VITE_SENTRY_DSN=...` (фронтенд);
- 3) для бэкенда достаточно `docker compose up -d backend` (обычная переменная окружения контейнера);
- 4) для фронтенда **обязательна пересборка**, а не просто перезапуск – значение запекается в собранный бандл на этапе `npm run build`: `docker compose up --build -d frontend`.

#### 4.4 Управление переменными окружения

Переменные окружения задаются в двух разных файлах, которые легко перепутать:

- `.env` в корне проекта (рядом с `docker-compose.yml`) – читается Docker Compose, используется при развёртывании через Docker (см. 3.1);
- `backend/.env` – читается бэкендом напрямую при запуске без Docker (нативный локальный запуск, см. README проекта, раздел «Вариант 2: локально»).

Полный перечень переменных, их назначение и значения по умолчанию – в приложении А.

#### 4.5 Безопасность инфраструктуры

- Оба прикладных контейнера (`backend`, `frontend`) работают от непривилегированного пользователя (не `root` внутри контейнера) – снижает ущерб при компрометации процесса внутри контейнера.

- Caddy – единственная точка входа снаружи (порты 80/443) и сам предоставляет security-заголовки ответа (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, Content-Security-Policy) – правка Caddyfile для этого не требуется.

- SECRET\_KEY используется для подписи сессионных токенов (JWT) – при его смене все ранее выданные токены перестают проходить проверку подписи, и все пользователи мгновенно разлогиниваются. Меняйте его осознанно (например, при подозрении на утечку), а не рутинно.

- SETUP\_TOKEN нужен только один раз, при первом запуске (см. 3.1); после того как первый владелец сети создан, его значение уже ни на что не влияет – эндпоинт /setup самоблокируется независимо от него.

- Система рассчитана ровно на один экземпляр контейнера backend (см. 2, «Условия применения») – не запускайте несколько его реплик одновременно, это нарушит защиту от перебора паролей, ограничение частоты запросов и рассылку напоминаний (все три реализованы в памяти процесса, без внешнего общего хранилища).

## 5 Сообщения оператору

В этом разделе приведены сообщения и ситуации, с которыми администратор сталкивается при развёртывании и эксплуатации сервера, их значение и рекомендуемые действия. Процедуры ниже рассчитаны на норматив времени восстановления после сбоя – не более 30 минут (ТЗ, раздел 4.1.3).

Сообщения при запуске и настройке

Таблица 1 – Сообщения при первом запуске и настройке окружения

| Сообщение                                                                                                                                                                                | Значение и действия                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SECRET_KEY не задан:<br>при DEBUG=false задайте<br>собственный<br>SECRET_KEY через<br>переменную окружения<br>или backend/.env                                                           | Контейнер backend отказывается стартовать<br>(сообщение попадает в журнал контейнера).<br>Заполните SECRET_KEY в .env и<br>перезапустите: docker compose up -d backend.                                                  |
| SETUP_TOKEN не задан:<br>при DEBUG=false задайте<br>SETUP_TOKEN через<br>переменную окружения<br>или backend/.env -- иначе<br>/api/setup останется<br>открытым для первого<br>встречного | Тот же случай для кода первого запуска –<br>заполните SETUP_TOKEN и перезапустите<br>контейнер.                                                                                                                          |
| Задайте SECRET_KEY<br>(python -c "import secrets;<br>print(secrets.token_urlsafe(<br>48))") / Задайте<br>SETUP_TOKEN (...)                                                               | Более ранняя проверка – сам Docker Compose<br>отказывается запускать контейнер backend,<br>если переменная не задана в .env вовсе (до<br>того, как это заметило бы само приложение).<br>Действие то же – заполнить .env. |

Продолжение таблицы 1

| Сообщение                            | Значение и действия                                                                                                                                                                                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Неверный код настройки (SETUP_TOKEN) | Значение, введённое в визарде /setup, не совпадает со значением SETUP_TOKEN в .env сервера. Сверьте значение в файле .env на сервере (не в примере/памяти) и введите его точно.                                               |
| Настройка уже выполнена              | Мастер /setup уже был пройден ранее (в системе уже есть владелец сети или администратор) – повторно создать первого владельца нельзя. Если нужно начать заново на новой инсталляции – используйте новую (пустую) базу данных. |

Сообщения при развёртывании, обновлении и работе контейнеров

Таблица 2 – Сообщения и ситуации при эксплуатации Docker-стека

| Ситуация                                                           | Значение и действия                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| git pull --ff-only<br>завершается ошибкой в<br>scripts/deploy.sh   | На сервере есть локальные изменения или расхождение истории с удалённым репозиторием (не должно происходить в норме, если код на сервере не редактируется вручную). Разберитесь с расхождением вручную (git status, git log) – не используйте принудительные операции без понимания причины. |
| scripts/deploy.sh<br>завершается с ненулевым<br>кодом после сборки | Один или несколько сервисов не стали healthy за отведённое время (120 секунд). Посмотрите docker compose ps и docker compose logs <сервис>, чтобы понять, какой сервис не поднялся и почему.                                                                                                 |

Продолжение таблицы 2

| Ситуация                                                                                   | Значение и действия                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 503 {"detail": "database unavailable"} на /health                                          | Бэкенд не может выполнить запрос к базе данных. Проверьте docker compose ps db и docker compose logs db.                                                                                                                                          |
| Контейнер backend не становится healthy                                                    | Дождитесь окончания применения миграций базы данных при первом старте (может занимать заметное время) – таймаут healthcheck на этот случай увеличен. Если проблема не в этом – смотрите docker compose logs backend.                              |
| Кнопка «Войти через VK» не появляется на сайте                                             | Это ожидаемое поведение при незаполненном VK_CLIENT_ID, а не ошибка – функция опциональна (см. 3.3).                                                                                                                                              |
| Письма (подтверждение почты, восстановление пароля, напоминания) не доходят                | Приложение не считает это ошибкой уровня сервиса – при сбое SMTP попытка отправки молча логируется, а не прерывает запрос пользователя. Проверьте docker compose logs backend на предмет ошибок SMTP и правильность заполнения SMTP_* переменных. |
| Ошибка CREATE EXTENSION / отсутствие расширения btree_gist (только при запуске без Docker) | У пользователя PostgreSQL, под которым выполняются миграции, нет прав на создание расширений. Выполните CREATE EXTENSION btree_gist; от имени суперпользователя PostgreSQL один раз, либо выдайте нужные права.                                   |

Действия при сбоях, подозрении на несанкционированный доступ и восстановлении

– **Сайт полностью недоступен.** Проверьте сначала доступность самого сервера (сеть, провайдер VPS), затем – docker compose ps на сервере.

Если один из сервисов не запущен – изучите его журнал (docker compose logs) и запустите заново (docker compose up -d).

- **Длительный отказ технических средств (сервер недоступен продолжительное время).** Разверните систему заново на новом сервере по 3.1 и восстановите базу данных из последней резервной копии по 4.2 – при условии, что копии предварительно выгружались за пределы исходного сервера (S3-бэкап либо ручная выгрузка ./backups).

- **Подозрение на несанкционированный доступ к серверу или учётным записям пользователей.** На уровне приложения смените SECRET\_KEY (мгновенно аннулирует все выданные сессии всех пользователей, см. 4.5) и SETUP\_TOKEN. Дополнительно проверьте на уровне самого сервера: список SSH-ключей и пользователей ОС, правила брандмауэра, журналы доступа – это выходит за рамки настоящей системы и относится к общей защите сервера.

- **Восстановление программ.** Код приложения восстанавливается повторным разворачиванием из репозитория (git clone/git pull + docker compose up --build -d) – сам код не хранит изменяемого состояния, вся эксплуатационная информация (пользователи, записи, услуги, настройки сайта, загруженные изображения) находится в базе данных и в томе uploads, которые резервному копированию (4.2) подлежат отдельно.

## ПРИЛОЖЕНИЕ А

### Переменные окружения

Переменные ниже задаются в корневом `.env` (Docker Compose, раздел 3.1) либо в `backend/.env` (нативный запуск без Docker) – файл указан в квадратных скобках в начале описания. Полный список со всеми техническими деталями и значениями по умолчанию для нативного запуска – всегда файл `backend/app/config.py` в исходном коде; здесь – практический минимум для администратора.

Таблица 3 – Переменные окружения

| Переменная        | Назначение и значение по умолчанию                                                                                                                                                          |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SECRET_KEY        | [оба] Секрет подписи JWT-токенов. Обязателен вне режима отладки. Без значения по умолчанию, генерируется командой (см. 3.1).                                                                |
| SETUP_TOKEN       | [оба] Код первого запуска для <code>/setup</code> (см. 3.1, 4.5). Обязателен вне режима отладки.                                                                                            |
| DEBUG             | [backend] <code>true</code> – режим разработки (авто-перезапуск, разрешён дефолтный секрет); <code>false</code> – прод-режим. В <code>docker-compose.yml</code> всегда <code>false</code> . |
| POSTGRES_PASSWORD | [корневой] Пароль пользователя PostgreSQL внутри docker-сети. По умолчанию <code>barbershop</code> – менять необязательно для одиночного сервера (порт базы наружу не публикуется).         |
| DATABASE_URL      | [backend] Строка подключения к PostgreSQL – только для нативного запуска; в Docker Compose собирается автоматически из <code>POSTGRES_PASSWORD</code> .                                     |
| SITE_ADDRESS      | [корневой] Домен сайта для Caddy. Не задано либо <code>:80</code> – обычный HTTP по IP.                                                                                                     |

Продолжение таблицы 3

| Переменная                                                                                          | Назначение и значение по умолчанию                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| COOKIE_SECURE                                                                                       | [оба] Флаг Secure у cookie с токеном сессии. Включайте true только тогда, когда перед приложением реально стоит HTTPS (Caddy с доменом) – по умолчанию false.                                                        |
| FRONTEND_BASE_URL                                                                                   | [оба] Базовый адрес сайта, используется в ссылках писем (сброс пароля и т. п.). По умолчанию http://localhost.                                                                                                       |
| ACCESS_TOKEN_EXPIRE_MINUTES                                                                         | [backend] Время жизни короткоживущего токена доступа. По умолчанию 15 минут – пользователь этого не замечает, токен обновляется автоматически через refresh-сессию (30 дней, не настраивается переменной окружения). |
| LOGIN_MAX_FAILED_ATTEMPTS / LOGIN_LOCKOUT_MINUTES                                                   | [backend] Блокировка входа после N неудачных попыток на M минут. По умолчанию 5 попыток / 15 минут.                                                                                                                  |
| PASSWORD_RESET_MAX_REQUESTS_PER_HOUR                                                                | [backend] Лимит запросов восстановления пароля. По умолчанию 3/час.                                                                                                                                                  |
| PASSWORD_RESET_TOKEN_EXPIRE_MINUTES                                                                 | [backend] Срок действия ссылки восстановления пароля. По умолчанию 30 минут.                                                                                                                                         |
| SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD / SMTP_USE_TLS / SMTP_FROM_EMAIL / SMTP_FROM_NAME | [оба] Параметры почтового сервера для исходящих писем (см. 3.3). Дефолты нацелены на локальный Mailpit.                                                                                                              |
| VK_CLIENT_ID / VK_CLIENT_SECRET / VK_REDIRECT_URI                                                   | [оба] Параметры входа через VK ID (см. 3.3). Без VK_CLIENT_ID функция отключена, это штатный режим.                                                                                                                  |

Продолжение таблицы 3

| Переменная                                                                                                       | Назначение и значение по умолчанию                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CORS_ORIGINS                                                                                                     | [оба] Список разрешённых источников для браузерных запросов (JSON-массив строк). В типовой топологии (Caddy → frontend → backend через один и тот же источник) фактически не задействован – актуален только при нестандартной топологии (например, API на отдельном поддомене). |
| SENTRY_DSN                                                                                                       | [backend] DSN проекта Sentry для мониторинга ошибок бэкенда (см. 4.3). Пусто – мониторинг выключен.                                                                                                                                                                             |
| VITE_SENTRY_DSN                                                                                                  | [сборка frontend] Тот же DSN для фронтенда – запекается в бандл на этапе сборки, требует docker compose up --build -d frontend при изменении.                                                                                                                                   |
| S3_BACKUP_ENDPOINT / S3_BACKUP_REGION / S3_BACKUP_BUCKET / S3_BACKUP_ACCESS_KEY_ID / S3_BACKUP_SECRET_ACCESS_KEY | [корневой] Параметры дополнительного резервного копирования в S3-совместимое хранилище (см. 4.2). Без них профиль s3-backup не запускается.                                                                                                                                     |

## ПРИЛОЖЕНИЕ Б

### Состав контейнеров

Таблица 4 – Сервисы docker-compose.yml

| Сервис       | Порты наружу | Назначение                                                                                                                                |
|--------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| db           | нет          | PostgreSQL 16 – основное хранилище данных.                                                                                                |
| backend      | нет          | FastAPI-приложение (порт 8000 виден только внутри docker-сети); применяет миграции при старте.                                            |
| frontend     | нет          | nginx: отдаёт собранное веб-приложение (SPA) и проксирует запросы /api/* на backend (порт 8080 виден только внутри docker-сети).          |
| caddy        | 80, 443      | Единственная точка входа снаружи: TLS-терминация, автоматический сертификат Let's Encrypt, security-заголовки, реверс-прокси на frontend. |
| backup-local | нет          | Ежедневный сжатый pg_dump в ./backups на диске сервера, с ротацией (4.2). Включён всегда.                                                 |
| backup-s3    | нет          | Тот же дамп в S3-совместимое хранилище. Выключен по умолчанию (профиль s3-backup).                                                        |

## ПРИЛОЖЕНИЕ В

### Глоссарий терминов

Таблица 5 – Термины, используемые в настоящем руководстве

| Термин         | Значение                                                                                                                                                                                |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Реверс-прокси  | Сервер (в этой системе – Caddy, а также nginx внутри frontend), принимающий внешние запросы и перенаправляющий их нужному внутреннему сервису.                                          |
| TLS-терминатор | Компонент, на котором заканчивается зашифрованное HTTPS-соединение и начинается обычный внутренний трафик – в этой системе им является Caddy.                                           |
| Healthcheck    | Периодическая проверка Docker’ом того, что контейнер не просто запущен, а действительно исправно отвечает (например, через GET /health у бэкенда).                                      |
| Том (volume)   | Место хранения данных контейнера, переживающее его пересоздание (например, pgdata для базы данных, uploads для загруженных изображений).                                                |
| Bind-mount     | Каталог на диске сервера, подключённый внутрь контейнера напрямую (в отличие от именованного тома) – в этой системе так устроен каталог ./backups, чтобы копии были видны и без Docker. |
| Миграция       | Управляемое изменение структуры базы данных (инструмент Alembic), применяется автоматически при каждом старте backend.                                                                  |

Продолжение таблицы 5

| Термин    | Значение                                                                                                                                                                                                    |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --ff-only | Опция git pull, разрешающая только перемотку вперёд (fast-forward) – команда завершится ошибкой при любом расхождении истории, вместо слияния вслепую.                                                      |
| Fail-fast | Принцип, при котором приложение отказывается стартовать при небезопасной конфигурации (например, без SECRET_KEY/SETUP_TOKEN вне режима отладки), вместо того чтобы молча работать в небезопасном состоянии. |

## ПРИЛОЖЕНИЕ Г

### Пример развёртывания на новом сервере

Сквозной пример – путь от пустого VPS до работающего сайта с доменом.

- 1) Арендован новый VPS с Linux, куплен домен `saitama-example.ru`, А-запись домена указывает на IP сервера.
- 2) На сервере установлен Docker (3.1, шаг 1), клонирован репозиторий, создан `.env` из примера.
- 3) Сгенерированы и вписаны в `.env` значения `SECRET_KEY` и `SETUP_TOKEN` (3.1, шаг 3).
- 4) Открыты порты в брандмауэре (3.1, шаг 4).
- 5) Выполнено `docker compose up --build -d`; через `docker compose ps` подтверждено, что все сервисы – `healthy` (3.5).
- 6) На `http://<IP>/setup` введён код настройки, создан первый владелец сети и точка «Сайтама на Тверской» (3.4).
- 7) В `.env` прописаны `SITE_ADDRESS=saitama-example.ru`, `COOKIE_SECURE=true`, `FRONTEND_BASE_URL=https://saitama-example.ru`; выполнено `docker compose up -d` – Caddy автоматически выпустил сертификат (3.2). Сайт теперь открывается по HTTPS.
- 8) Заполнены `SMTP_*` для боевого почтового провайдера (3.3) – проверено восстановление пароля тестового аккаунта, письмо получено.
- 9) Через месяц эксплуатации вышло обновление кода – на сервере выполнено `./scripts/deploy.sh`, скрипт подтвердил успешный статус всех сервисов (4.1).
- 10) Раз в неделю администратор копирует каталог `./backups` на другую машину (`rsync`) в дополнение к автоматической локальной ротации (4.2).